Forecast Verification (ngen-verf)

Table of Contents

Introduction

ngen-verf is a command-line tool designed for verifying NWM streamflow forecasts across short-range, medium-range, and long-range configurations. It can also be used to compare verification statistics between different forecast datasets, whether from different NWM versions or forecast periods.

The **ngen-verf** workflow consists of five sequential steps:

1. Fetch NWM Streamflow Forecast Data

In this step, NWM streamflow forecast data are downloaded based on the specified configuration, versions, and time periods defined in the YAML configuration file. The data are then extracted for the designated locations. **ngen-verf** leverages the Python library **TEEHR** (Tools for Explanatory Evaluation in Hydrologic Research) to retrieve operational NWM forecasts from Google Cloud Storage (GCS) and store them as Parquet files in the TEEHR data model.

*This step can be highly resource-intensive and time-consuming. Please refer to the **Tips and Troubleshooting** section for guidance on optimizing performance.*

2. Fetch Streamflow Observation Data

In this step, streamflow observations are downloaded for the specified time periods and locations as defined in the YAML configuration file. Currently, **ngen-verf** supports only **USGS streamflow observations**.

3. Pair Observations with Forecast Data

Next, streamflow forecasts are paired with corresponding observations based on validation time and location. This step requires both forecast and observation datasets to be fully downloaded and processed in the preceding steps.

4. Compute Evaluation Metrics

Once forecasts and observations are paired, **ngen-verf** calculates performance evaluation metrics for the lead times and statistics defined in the YAML configuration file. These computations are based on the paired datasets from the previous step.

5. Generate Visualization of Metrics

In the final step, visualizations are created for the lead times and metrics specified in the YAML configuration file. **ngen-verf** produces three types of plots:

- **Spatial maps** – displaying geographic patterns in forecast performance
- **Histograms** – illustrating the distribution of metric values
- **Boxplots** – summarizing metric variability across different conditions

Installation & Usage

Run ngen-verf from docker container

1. Log into docker with your credentials

```
docker login registry.sh.nextgenwaterprediction.com
```

2. Pull the latest ngen-verf docker image

```
docker pull registry.sh.nextgenwaterprediction.com/ngwpc/nwm-ngen/ngen-verf:latest && docker tag registry.sh.nextgenwaterprediction.com/ngwpc/nwm-ngen/ngen-verf:latest ngen-verf
```

3. Start an ngen-verf container

docker run -v [DIR_LOCAL]:[DIR_CONTAINER] -it --entrypoint /bin/bash ngen-verf

#The following example command will launch an ngen-cerf container, mapping my home directory to the same directory within the container, so that I can always access all data stored in my home directory in the same way, regardless whether I run verification locally or from a container.

docker run -v /home/yuqiong.liu:/home/yuqiong.liu -it --entrypoint /bin/bash ngen-cal

4. Set up configuration for verification

Make a copy of the example configuration file (/ngen-app/ngen-verf/sample_files/config.yaml) and adjust the contents as need to set up your configuration. Sample gage files needed for the file_paths section of the configuration file can be found in /ngen-app/ngen-verf/sample_files.

5. Run the verification application

python /ngen-app/ngen-verf/verification.py config.yaml

6. Repeat steps 4 & 5 to run another verification application

Clone, build, and run ngen-verf locally

Clone ngen-verf from Gitlab

```
cd [NGEN_VERF_ROOT]
git clone -b development --recurse-submodules https://gitlab.sh.nextgenwaterprediction.com/NGWPC/nwm-ngen/ngen-verf.git
```

Clone ngen-eval from Gitlab

ngen-verf requires ngen-eval as a dependency

```
cd [NGEN_EVAL_ROOT]
git clone -b development --recurse-submodules https://gitlab.sh.nextgenwaterprediction.com/NGWPC/nwm-ngen/ngen-eval.git
```

Create python venv

```
cd [VENV_ROOT]
/usr/bin/python3.11 -m venv venv
source venv/bin/activate
pip install --upgrade pip
```

Install ngen-eval

```
cd [NGEN_EVAL_ROOT]/ngen-eval
pip install .
```

Install ngen-verf

```
cd [NGEN_VERF_ROOT]/ngen-verf
pip install .
```

where [NGEN_EVAL_ROOT], [NGEN_VERF_ROOT], [VENV_ROOT] refer to the directory to install ngen-eval, ngen-verf, and python venv in your local workspace, respectively.

Run ngen-verf locally

1) Set up your verification application via a configuration yaml file (e.g., config.yaml)

Follow the sample config file (ngen-verf/sample_files/config.yaml) and the comments therein to set up the configurations for your verification application as needed.

2) run the verification script with the configuration

```
python [NGEN_VERF_ROOT]/ngen-verf/verification.py config.yaml
```

3) Repeat the first two steps as many times as needed

Configure verification applications

Sample config file

```
#### configuration for the verification run
general:

  # define which of the 5 steps to run; each step can be run independently,
  # assuming data from previous steps (from previous runs) are available for use
  steps:
    fetch_fcst_data: True
    fetch_obs_data: True
    pair_data: True
    compute_metrics: True
    plot_metrics: True

  # user-specified name for the set of locations to carry out verification for
  # this name is used to create a folder in 'data_dir_root' (in 'file_paths' section) to store all the datasets
and outputs
  # see function 'data_paths' in ngen-verf/src/ngen/verf/settings.py to see how the overall data directory
structure is defined
  location_set_name: "calib_basin_group3"

  # [optional] but either location_list or location_list_file (in ['file_paths']) must be specifed
  # list of usgs gage IDs or NWM links IDs; if blank, a location_list_file must be provided in 'file_paths'
section
  location_list:

  # [optional] but must be specified if locaiton_list is not blank; currently supported options: usgs_gage,
nwm30_link or nwm22_link
  # depending on which NWM versions are listed in ['general'] ['nwm_version']
  location_type:

  variable_name: streamflow # currently only option is 'streamflow'
  nwm_configuration:  short_range  # e.g., analysis_assim, short_range, analysis_assim_hawaii, medium_range_mem1

  # user-specified name of datasets for comparison (to be used for creating sub-folders for datasets and plots)
  dataset_name: ['v3_sep', 'v3_oct']

  # list of NWM versions; list must have the same length as 'dataset_name'
  # Currently accepts "nwm22" or "nwm30"; Use "nwm22" for dates prior to 09-19-2023
  nwm_version: ['nwm30', 'nwm30']

  # list of start/end dates for forecast verification period; list must have the same length as 'dataset_name'
  forecast_start_date: ['2024-09-01', '2024-10-01']
```

```
  forecast_end_date: ['2024-09-30', '2024-10-31']

#### configurations for various file paths
# see some sample files at ngen-verf/sample_files/gage_files
file_paths:

  # root directory to store the downloaded NWM forecast and flow observation data
  data_dir_root: /home/yuqiong.liu/work/data/ngen-verf

  # [optional] but either location_list (in ['general']) or location_list_file must be specifed
  # tab or comma delimited csv file, which must contain either a "gage" or
  # a "link" column specifying the locations to conduct verification for, with NWM version indicated in column
name (e.g., nwm30_link)
  location_list_file: /home/yuqiong.liu/work/data/ngen-verf/usgs_gages_link_CONUS_calib100.csv

  # parquet files containing the crosswalk between NWM feature_id and usgs gage id
  # note a crosswalk file should be provided for each NWM version listed in ['general']['nwm_version']
  # see utils/create_cralib_basin_group1osswalk.py on how the sample crosswalk parquet was created
  crosswalk_file:
    nwm30: /home/yuqiong.liu/work/data/ngen-verf/crosswalks/usgs_nwm30_crosswalk_CONUS.parquet
    #nwm22:

  # meta data for USGS gages
  gage_meta_file: /home/yuqiong.liu/work/data/ngen-verf/gages_metadata_all_domains.csv

  # geometry (point lat/lon) for USGS gages; see utils/create_geometry.py on how the sample geometry parquet
was created
  geometry_file: /home/yuqiong.liu/work/data/ngen-verf/geometry/usgs_point_geometry_CONUS.parquet

#### configurations for retrieving NWM forecast data (typically no need to change)
nwm_forecast:
  fetch_fcst: [True, True]  # whether to fetch the forecast data for each dataset listed in ['general']
['dataset_name']
  output_type: channel_rt
  t_minus: [0, 1, 2]  # Only used if an assimilation run is selected
  data_source: GCS    # Specifies the remote location from which to fetch the data ("GCS", "NOMADS", "DSTOR")
  kerchunk_method: local  # When data_source = "GCS", specifies the preference in creating Kerchunk reference
json files.
                          # "local" - always create new json files from netcdf files in GCS and save locally,
if they do not already exist
                          # "remote" - read the CIROH pre-generated jsons from s3, ignoring any that are
unavailable
                          # "auto" - read the CIROH pre-generated jsons from s3, and create any that are
unavailable, storing locally

  process_by_z_hour: True  # If True, NWM files will be processed by z-hour per day. If False, files will be
                         # processed in chunks (defined by STEPSIZE). This can help if you want to read many
reaches
                         # at once (all ~2.7 million for medium range for example).

  stepsize: 100  # Only used if PROCESS_BY_Z_HOUR = False. Controls how many files are processed in memory at
once
                # Higher values can increase performance at the expense on memory  (default value: 100)

  ignore_missing_file: False  # If True, the missing file(s) will be skipped and the process will resume
                           # If False, TEEHR will fail if a missing NWM file is encountered

  overwrite_output: False  # If True, existing output files will be overwritten; if False, existing files are
retained

#### configurations for retrieving streamflow observation data (currently only usgs is supported)
#### typically no need to change selections in this section
flow_observation:
  usgs: # flow obs retrieved will be stored at [data_dir_root]/[location_set_name]/usgs
    chunk_by: month   # data are saved in parquet files by month
    overwrite_output: False  # If True, existing output files will be overwritten; if False, existing files are
retained

#### paired data generation
pair_data:
```

```
  # whether to overwrite existing joined parquet files
  overwrite: True

#### configuration for metrics calculation
metrics:

  # whether to overwrite existing metric files
  overwrite: True

  # library to be used for metric calculation; currently supported options: teehr, ngen.eval
  # see the lists of supported metrics toward the end of sample config.yaml file (also ngen-verf/src/ngen/verf
/settings.py)
  library: 'ngen.eval'

  # list of metrics to be calculated
  # if set to 'all', all available metrics will be calculated
  # [caution] some metrics available in ngen.eval (e.g., event-based metrics) may be time consuming to compute
  metric_subset: ['KGE','NSE','CORR','NNSE']

  # non-exceedance probability threshold of streamflow to be used for categorical metrics in ngen.eval
  flow_threshold_categorical: 0.9

  # non-exceedance probability threshold of streamflow to be used for event-based metrics in ngen.eval
  flow_threshold_event: 0.9

  # list of lead times (in hours) for which metrics should be calculated for
  # the lead times can be integers (e.g., 1,2), or an interval (e.g., 1-3), or 'all'
  # if set to 'all', all raw lead times (i.e., 1, 2, ..., 18 in the case of short-range forecasts) will be
calculated
  #lead_times: [1,3,5,'1-3','1-5','4-6','6-10']
  lead_times: ['all','1-5']

#### configuration for plots
# For each type of plots (histogram, boxplot, spatial map),
#   1) 'metric_subset' must be a subset of metrics defined for calculation (i.e., 'metric_subset' defined in
Section 'metrics')
#   2) 'lead_times' must be a subset of lead times defined for metrics calculation (i.e., 'lead_times' defined
in Section 'metrics')
# For sample plots, check out sample_files/metric_plots
plots:
  histogram:

    # whether to create histogram plots
    plot: True

    # list of metrics to create histogram plots for
    metric_subset: ['KGE','NSE','CORR','NNSE']

    # for each metric, define bins for generating the histogram plot
    # For 'KGE','NSE','CORR' and 'NNSE', default bins will be used if not defined.
    # For other metrics, bins must be specified here
    binning:
      KGE: [-inf, -1,-0.5,0,0.2,0.4,0.6,0.8,1.0]
      NSE: [-inf, -1,-0.5,0,0.2,0.4,0.6,0.8,1.0]
      NNSE: [0,0.2,0.4,0.5,0.6,0.7,0.8,1.0]
      CORR: [-1,-0.5,0,0.2,0.4,0.6,0.8,1.0]

    # list of lead times (in hours) to create histograms for
    lead_times: [1,3,5,10,'1-5']

    # define a string that gets added to the filename of histogram plots to differentiate between plots
    # of different lead time groups or different binning scenarios, to avoid overwriting the existing plots.
    # Leave it blank if overwriting is desired
    tag: test1

  boxplot: # the same comments for histogram apply here (except binning)
    plot: True
    metric_subset: ['KGE','NSE','CORR','NNSE']
    lead_times: [1,3,5,10,1-5]
    tag: test1
```

```
  spatial_map:
    plot: True

    # list of metrics to create spatial maps for
    metric_subset: ['KGE','NSE','CORR','NNSE']

    # for each metric, define an appropriate range (mix/max) to scale the spatial map
    # For 'KGE','NSE','CORR' and 'NNSE', default ranges will be used if not defined.
    # For other metrics, a range must be specified here
    scaling:
      KGE: [-0.5,1]
      NSE: [-0.5,1]
      CORR: [-0.5,1]
      #NNSE: [0.2, 0.7]

    #lead_times: [1,3,5,10]
    lead_times: [1, 3, 5, 10, 1-5]

    # define a string that gets added to the filename of spatial map plots to avoid overwriting the existing
ones (if any)
    tag: test1
```

Explanation for section 1: **general**

Field	Nested field if any	Optional?	Value Type	Sample Value	Notes
steps	fetch_fcst_data	No	Boolean, True or False	True	Whether to fetch NWM forecasts in the current run; setting to False to skip the step
	fetch_obs_data	No	Boolean, True or False	True	Whether to fetch USGS streamflow observations in the current run; setting to False to skip the step
	pair_data	No	Boolean, True or False	True	Whether to join the time series of NWM forecasts and observation data based on location and valid time in the current run
	compute_metrics	No	Boolean, True or False	True	Whether to compute metrics in the current run
	plot_metrics	No	Boolean, True or False	True	Whether to create plots of metrics in the current run
location_set_name		No	String	calib_basin_group1	User-specified name for the set of locations to carry out verification for. This string is used to create a folder in **data_dir_root** (in **file_paths** section) to store all the datasets and outputs. See the Verification datasets and outputs section for details the overall data directory structure
location_list		Yes	List of strings or integers	[01123000, 01010000] or [8134650, 23963741]	List of usgs gage IDs (strings) or NWM links IDs (integers). Number of items in the list can be any integer >= 1. Note for list of strings, it is optional to put quotation marks around each string, i.e., both [01123000, 01010000] and ['01123000', '01010000'] Either **location_list** or **location_list_file** (in **file_paths** section) must be specified. If both are defined, **location_list** will take priority
location_type		Yes	String	usgs_gage, or nwm30_link, or nwm22_link	Must be specified if **location_list** is not blank; currently supported options: usgs_gage, nwm30_link or nwm22_link, indicating whether the location IDs specified in **location_list** is USGS gages, or NWM v30 feature (or link) IDs, or NWM v22 link IDs.
variable_name		No	String	streamflow	Currently only option is streamflow
nwm_configuration		No	String	short_range	String indicating the NWM configuration to conduct verification for. Example of valid options: analysis_assim, short_range, analysis_assim_hawaii, medium_range_mem1
dataset_name		No	List of strings	['v3_sep', 'v3_oct']	User-specified names for the datasets to conduct verification for and to be compared with each other. These strings will be used in the names of the data directories or files or plots. See the Verification datasets and outputs section. The number of datasets can be any integer >= 1
nwm_version		No	List of strings	['nwm30', 'nwm30']	List of strings indicating NWM versions; list must have the same length as **dataset_name**. Currently accepts "nwm22" or "nwm30"; Use "nwm22" for dates prior to 09-19-2023
forecast_start_date		No	List of datetime strings	['2024-09-01', '2024-10-01']	List of start dates for forecast verification period; List must have the same length as **dataset_name**

forecast_end_date	No	List of datetime strings	['2024-09-30', '2024-10-31']	List of end dates for forecast verification period; List must have the same length as **dataset_name**

Explanation for section 2: **file_paths**

Field	Nested field if any	Optional?	Value Type	Sample Value	Notes
data_dir_root		No	String	/home/yuqiong.liu/work/data/ngen-verf	Root directory to store the downloaded NWM forecast and flow observation data, as well as the paired data, the calculated metrics, and the plots generated. See the Verification dataset and output section for details the overall data directory structure. If this root directory does not already exist, it will be created.
location_list_file		Yes	FilePath	/home/yuqiong.liu/work/data/ngen-verf/usgs_gages_link_CONUS_calib100.csv	Path to a tab or comma delimited csv file, which must contain either a "gage" or "link" column specifying the locations to conduct verification for, with NWM version indicated in column name (e.g., nwm30_link) Either **location_list (**in **general** section) or **location_list_file** must be specified. If both are defined, **location_list** will take priority
crosswalk_file	nwm30	Yes	FilePath	/home/yuqiong.liu/work/data/ngen-verf/crosswalks/usgs_nwm30_crosswalk_CONUS.parquet	Only needed for NWM versions defined in **nwm_version** in the general section
	nwm22	Yes	FilePath		Only needed for NWM versions defined in **nwm_version** in the general section
gage_meta_file		No	FilePath	/home/yuqiong.liu/work/data/ngen-verf/gages_metadata_all_domains.csv	Path to a tab or comma delimited csv file containing some meta data information (e.g., agency, name, NWS ID etc) for each gage. Currently, this file is only used to determine whether a given gage ID is a USGS gage, requiring minimally a column "gage" and another column "agency". See utils/create_cralib_basin_group1_crosswalk.py on how the sample crosswalk parquet was created.
geometry_file		No	FilePath	/home/yuqiong.liu/work/data/ngen-verf/geometry/usgs_point_geometry_CONUS.parquet	Path to a parquet file containing the geometry (point lat/lon) for USGS gages; see utils/create_geometry.py on how the sample geometry parquet was created

Explanation for section 3: **nwm_forecast**

This section is used to set up configurations for retrieving and processing NWM forecast data using the TEEHR library. Typically there is no need to change entries in this section except for the first one.

Field	Nested field if any	Optional?	Value Type	Sample Value	Notes
fetch_fcst		No	List of Boolean	[True, True]	Whether to fetch the forecast data for each dataset listed in **dataset_name** in the general section. List must have the same length as **dataset_name**
output_type		No	String	channel_rt	Currently only option is 'channel_rt' (i.e., the streamflow simulation from NWM forecasts)
t_minus		No	List of integers	[0, 1, 2]	Only used if an assimilation run is selected
data_source		No	String	GCS	Specifies the remote location from which to fetch the data ("GCS", "NOMADS", "DSTOR")
kerchunk_method		No	String	local	When data_source = "GCS", specifies the preference in creating Kerchunk reference json files. "local" - always create new json files from netcdf files in GCS and save locally, if they do not already exist "remote" - read the CIROH pre-generated jsons from s3, ignoring any that are unavailable "auto" - read the CIROH pre-generated jsons from s3, and create any that are unavailable, storing locally
process_by_z_hour		No	Boolean	True	If True, NWM files will be processed by z-hour per day. If False, files will be processed in chunks (defined by STEPSIZE). This can help if you want to read many reaches at once (all ~2.7 million for medium range for example).
stepsize		No	Integer	100	Only used if PROCESS_BY_Z_HOUR = False. Controls how many files are processed in memory at once. Higher values can increase performance at the expense on memory (default value: 100)
ignore_missing_file		No	Boolean	True	If True, the missing file(s) will be skipped and the process will resume. If False, TEEHR will fail if a missing NWM file is encountered

overwrite_output	No	Boolean	False	If True, existing output files will be overwritten; if False, existing files are retained. It is recommended to set it to False to allow using existing parquet files that are previously downloaded and processed.

Explanation for section 4: **flow_observation**

This section is used to set up configurations for downloading streamflow observation data using the TEEHR library. Currently, only USGS streamflow data is supported. Typically there is no need to change entries in this section.

Field	Nested field if any	Optional?	Value Type	Sample Value	Notes
usgs	chunk_by	No	String	Month	Data are saved in parquet files by month (default)
	overwrite_output	No	Boolean	False	If True, existing output files will be overwritten; if False, existing files are retained

Explanation for section 5: **pair_data**

Currently only one field is needed for this section.

Field	Optional?	Value Type	Sample Value	Notes
overwrite	No	Boolean	False	If True, existing paired files will be overwritten; if False, existing files are retained

Explanation for section 6: **metrics**

The section is used to set up configurations for metrics calculation.

Field	Nested field if any	Optional?	Value Type	Sample Value	Notes
overwrite		No	Boolean	True	whether to overwrite existing metric files
library		No	String	ngen.eval	Python library to be used for metric calculation; currently supported options: teehr, ngen.eval. The list of supported metrics for each library can be found at the Metrics supported section. Note ngen.eval is built on the metric functions used by the calibration software ngen-cal.
metric_subset		Yes	List of strings	['KGE','NSE','CORR','NNSE']	List of metrics to be calculated. If set to 'all', all available metrics supported by the chosen library will be calculated Caution: some metrics available in ngen.eval (e.g., event-based metrics) may be time consuming to compute
flow_threshold_categorical		Yes	Float	0.9	Non-exceedance probability threshold of streamflow to be used for categorical metrics in ngen.eval. If not set, default to 0.9
flow_threshold_event		Yes	Float	0.9	Non-exceedance probability threshold of streamflow to be used for event-based metrics in ngen.eval. If not set, default to 0.9
lead_times		No	List of strings or integers	[1,3,5,'1-3','1-5','4-6','6-10']	List of lead times (in hours) for which metrics should be calculated for. The lead times can be integers (e.g., 1, 2), or an interval (e.g., 1-3), or 'all'. If set to 'all', all raw lead times (i.e., 1, 2, ..., 18 in the case of short-range forecasts) will be calculated. An interval based lead time (e.g., 1-3 hours) indicates calculating metrics using data pooled from a group of raw lead times (e.g., 1, 2, and 3).

Explanation for section 7: **plots**

The section is used to set up configurations for visualizing metrics and exported to png files. ngen-verf will generate three different types of plots: spatial maps, histograms, and boxplots.

1) For histogram, a plot comparing the datasets is generated for each lead time and each metric

2) For boxplot, a plot comparing the datasets and different lead times is generated for each metric

3) For spatial map, a plot is generated for each dataset, lead time, and metric

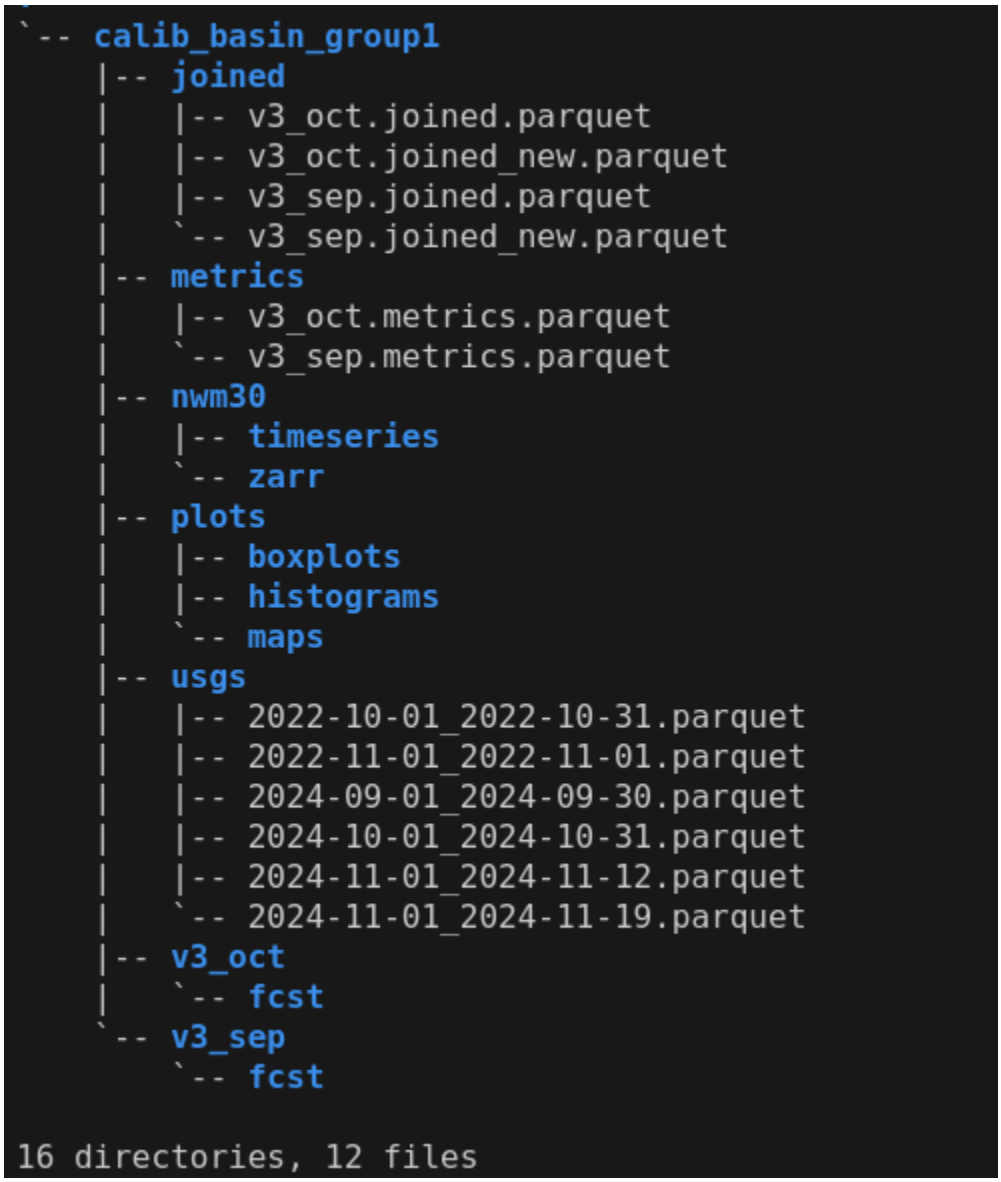
For sample plots in each category, check out Sample plots

Field	Nested field if any	Optional?	Value Type	Sample Value	Notes
histograms	plot	No	Boolean	True	Whether to create histogram plots

	metric_subset	No	List of strings	['KGE','NSE','CORR','NNSE']	List of metrics to create histogram plots for. This list must be a subset of the list of metrics calculated, as defined by **metrics_subset** in Metrics section
	binning	Yes	Dictionary	KGE: [-inf, -1, -0.5,0,0.2,0.4,0.6,0.8,1.0] NSE: [-inf, -1, -0.5,0,0.2,0.4,0.6,0.8,1.0] NNSE: [0,0.2,0.4,0.5,0.6,0.7,0.8,1.0] CORR: [-1, -0.5,0,0.2,0.4,0.6,0.8,1.0]	For each metric, define bins for generating the histogram plots. For 'KGE','NSE','CORR' and 'NNSE', default bins (as shown in "Sample Value" column) will be used if not defined. For other metrics, bins must be specified here.
	lead_times	No	List of integers or strings	[1,3,5,10,'1-5']	List of lead times (in hours) to create histograms for. Notes: 1) lead limes can be either selected raw lead times (e.g., 1, 2, ... , 18 for short-range forecasts) or grouped lead times (e.g., 1-5 hours), where selected raw lead times are grouped together to compute statistics 2) lead times must be a subset of lead times defined for metrics calculation by **lead_times** in Metrics section
	tag	Yes	String	test1	A string that gets added to the filename of histogram plots to differentiate between plots from verification runs with different configurations, e.g., different lead time groups or different binning scenarios, to avoid overwriting the existing plots. Leave it blank if overwriting is desired
boxplots	plot	No	Boolean	True	Whether to create boxplots
	metric_subset	No	List of strings	['KGE','NSE','CORR','NNSE']	List of metrics to create boxplots for. This list must be a subset of the list of metrics calculated, as defined by **metrics_subset** in Metrics section
	lead_times	No	List of integers or strings	[1,3,5,10,'1-5']	Same as for histograms
	tag	No	String	test	Same as for histograms
Spatial maps	plot	No	Boolean	True	Whether to create spatial maps
	metric_subset	No	List of strings	['KGE','NSE','CORR','NNSE']	List of metrics to create histogram plots for. This list must be a subset of the list of metrics calculated, as defined by **metrics_subset** in Metrics section
	scaling	Yes	Dictionary	KGE: [-0.5,1] NSE: [-0.5,1] CORR: [-0.5,1] NNSE: [0, 1]	For each metric to plot, define an appropriate range (mix/max) to scale the spatial map For 'KGE','NSE','CORR' and 'NNSE', default ranges will be used if not defined. For other metrics, a range must be specified here
	lead_times	No	List of integers or strings	[1,3,5,10,'1-5']	Same as for histograms
	tag	Yes	String	test1	Same as for histograms

Verification datasets and outputs

Directory structure



Notes on the Directory Structure

1. **`calib_basin_group1`** corresponds to the `dataset_name` field specified in the YAML configuration. It serves as the root directory, storing all datasets and outputs generated during the verification process.
2. **Step 1: Fetch NWM Forecast Data (`fetch_fcst_data`)**
    - Downloads and processes NWM forecast data for the specified locations and time periods.
    - Stores raw forecast data in the `nwm30/zarr` subdirectory.
    - Converts the data into Parquet format and saves it in `nwm30/timeseries`.
    - Creates symbolic links to the required Parquet files for each dataset (e.g., `v3_oct/fcst`, `v3_sep/fcst`).
3. **Step 2: Fetch Streamflow Observation Data (`fetch_obs_data`)**
    - Downloads USGS streamflow data.
    - Stores observations as Parquet files in the `usgs` subdirectory, with each month's data saved in a separate file.
4. **Step 3: Pair Forecast and Observation Data (`pair_data`)**
    - Generates paired datasets and saves them as Parquet files in the `joined` subdirectory (e.g., `v3_oct_joined.parquet`, `v3_sep_joined.parquet`).
    - *Note:* Additional files (e.g., `v3_oct_joined_new.parquet`, `v3_sep_joined_new.parquet`) are created during the next step (metric computation). These can be manually deleted afterward to free up storage if needed.
5. **Step 4: Compute Metrics (`compute_metrics`)**
    - Generates a metrics Parquet file for each dataset.
6. **Step 5: Plot Metrics (`plot_metrics`)**

- Creates three types of plots and saves them as PNG files in the following subdirectories:
  - **Boxplots:** `plots/boxplots`
  - **Histograms:** `plots/histograms`
  - **Maps:** `plots/maps`

Notes on NWM Forecast Data

- **Raw NWM Forecast Data**
  The downloaded NWM forecast data is stored in JSON format at:
  `[data_dir_root]/[location_set_name]/[nwm_version]/zarr/[nwm_configuration]`,
  with one file per lead time per forecast hour. Example:

  `/home/yuqiong.liu/work/data/ngen-verf/calib_basin_group1/nwm30/zarr/short_range/nwm.20241031.nwm.t23z.short_range.channel_rt.f018.conus.nc.json`

- **Processed NWM Forecasts**
  Forecast data for locations specified in either `location_list` or `location_list_file` is processed and stored in Parquet format at:
  `[data_dir_root]/[location_set_name]/[nwm_version]/timeseries/[nwm_configuration]`
  Example:

  `/home/yuqiong.liu/work/data/ngen-verf/calib_basin_group1/nwm30/timeseries/short_range/20241031T23.parquet`

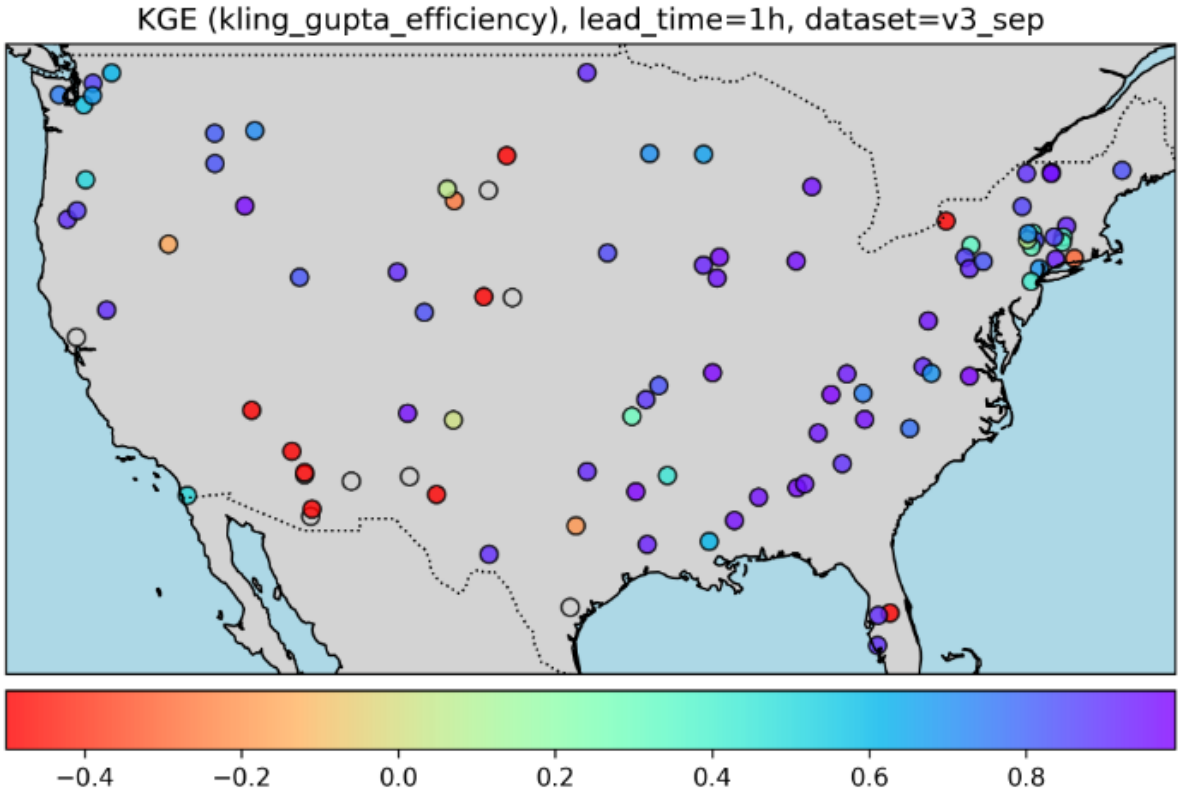
- **Dataset-Specific Storage and Symbolic Links**
  A subdirectory is created for each dataset under:
  `[data_dir_root]/[location_set_name]/[dataset_name]`
  Within this directory, symbolic links to the relevant Parquet files are generated. This allows datasets to selectively access only the required NWM forecasts, based on parameters such as `nwm_version`, `nwm_configuration`, `forecast_start_time`, and `forecast_end_time` specified in the dataset's configuration.

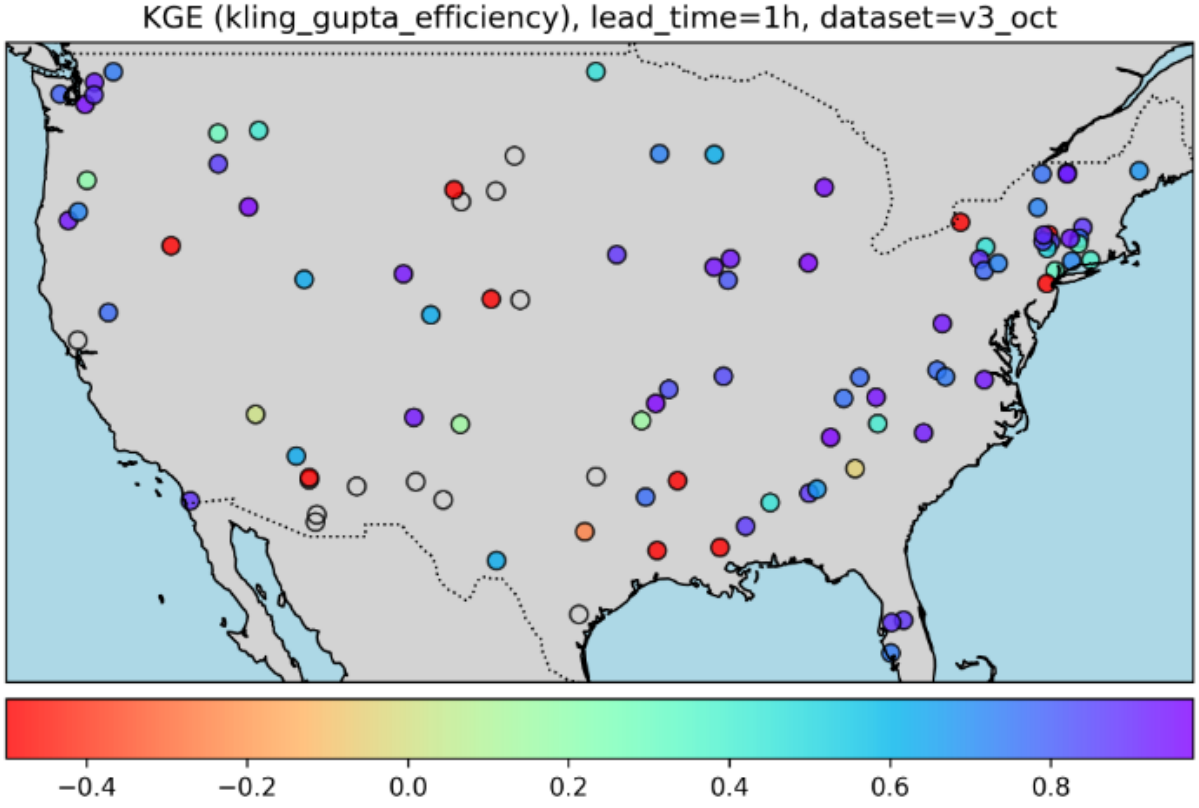
- **Efficient Data Handling**
  The symbolic link structure enables flexible dataset adjustments without requiring re-downloading or reprocessing of NWM forecast data, which can be resource-intensive. The subsequent **data pairing step (`pair_data`)** is conducted using these symbolic links, ensuring efficiency and minimizing redundant computations.

Sample plots

Sample spatial maps

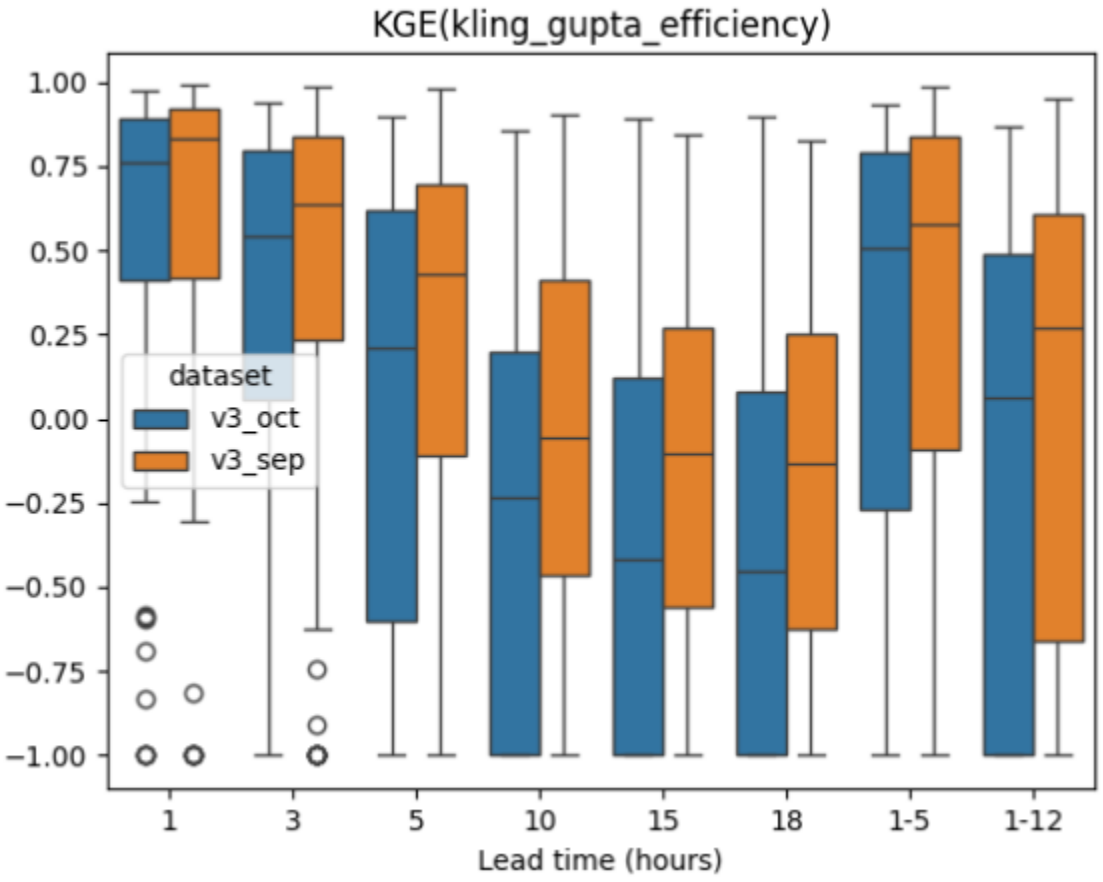
Spatial maps of KGE from the first lead hour comparing the two datasets (left PNG: v3_sep, right PNG: v3_oct)





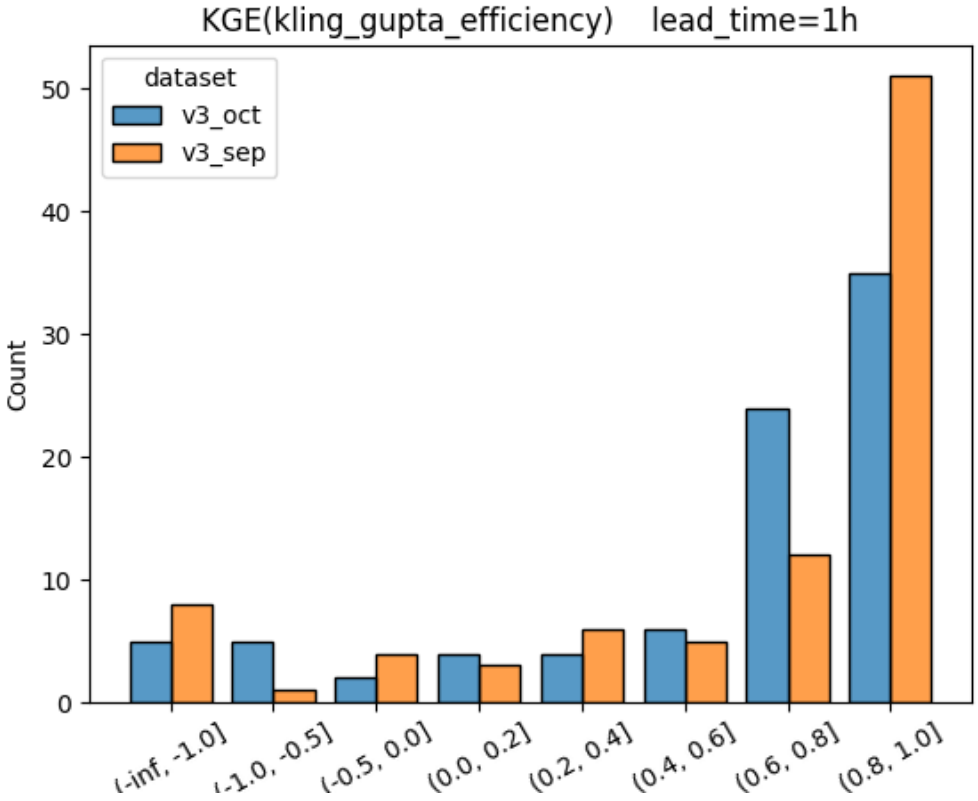
Sample boxplots

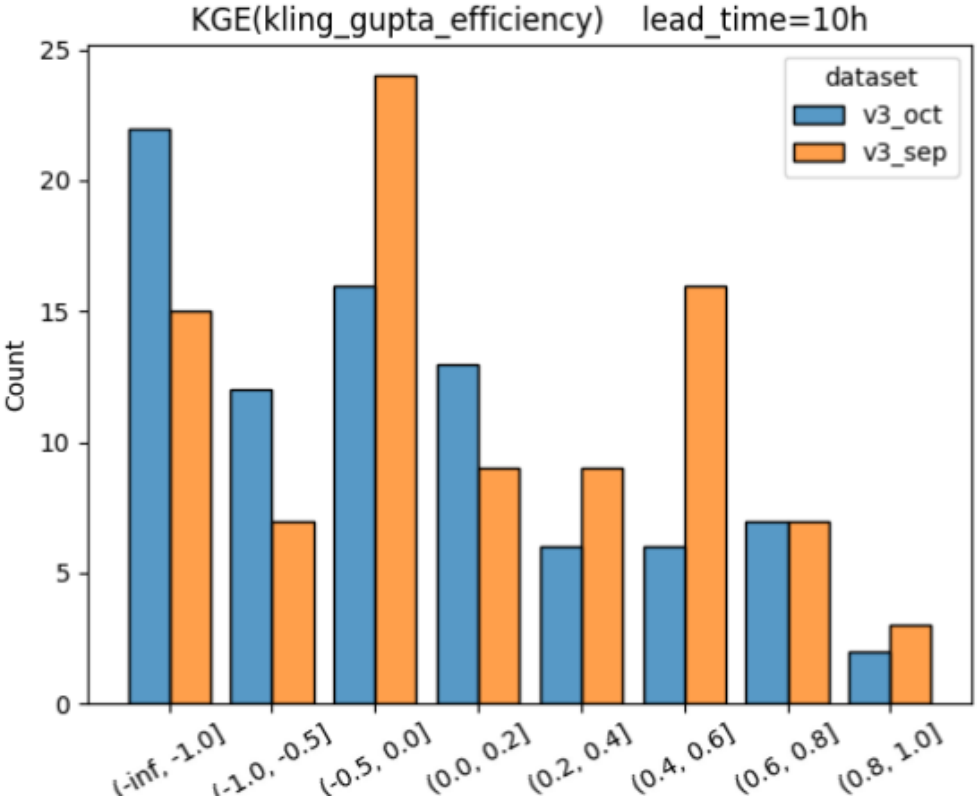
Boxplot summarizing KGE from all locations for selected lead times as defined in the configuration, comparing the two datasets (v3_sep vs v3_oct) side by side



Sample histograms

Histograms summarizing KGE from all locations for the two datasets (left PNG: lead_time=1h, right PNG: lead_time=10h)





Metrics Supported

ngen-verf currently supports metric calculation using either the TEEHR library or the ngen-eval library. The metrics supported by each library are listed below.

Metric Short Name	Metric Long Name	Supported by TEEHR?	Supported by ngen-eval?
CORR	Pearson correlation	Yes	Yes
sCORR	Spearman correlation	Yes	No
NSE	Nash Sutcliffe Efficiency	Yes	Yes
NNSE	Normalized Nash Sutcliffe Efficiency	Yes	Yes
NSElog	logrithmic Nash Sutcliffe Efficiency	No	Yes
NSEwt	weighted NSE and NSElog	No	Yes
KGE	Kling Gupta Efficiency	Yes	Yes
KGE1	Kling Gupta Efficiency mod1	Yes	No
KGE2	Kling Gupta Efficiency mod2	Yes	No
ME	mean error	Yes	No
MAE	mean absolute error	Yes	Yes
MSE	mean squared error	Yes	No
RMSE	root mean squared error	Yes	Yes
RMAE	mean absolute relative error	Yes	No
PBIAS	percent bias	No	Yes
RBIAS	relative bias	Yes	No
MBAIS	multiplicative bias	Yes	No
aprBIAS	annual peak relative bias	Yes	No
R2	r squared	Yes	No
RSR	RMSE observation standard deviation ratio	No	Yes
HSEG_FDC	percent bias in high flow section of flow duration curve	No	Yes
MSEG_FDC	percent bias in medium flow section of flow duration curve	No	Yes
LSEG_FDC	percent bias in low flow section of flow duration curve	No	Yes
POD	probability of detection	No	Yes
FAR	false alarm ratio	No	Yes
CSI	critical success index	No	Yes
FBIAS	frequency bias	No	Yes
PKBIAS	percent peak flow bias	No	Yes
PKTE	peak timing error	No	Yes
EVBIAS	event volume bias	No	Yes
n_obs	observation count	Yes	No
n_mod	simulation count	Yes	No
min_obs	observation minimum	Yes	No
min_mod	simulation minimum	Yes	No
max_obs	observation maximum	Yes	No
max_mod	simulation maximum	Yes	No
mean_obs	observation average	Yes	No
mean_mod	simulation average	Yes	No
sum_obs	observation sum	Yes	No
sum_mod	simulation sum	Yes	No
var_obs	observation variance	Yes	No
var_mod	simulaiton variance	Yes	No
max_delta	max value delta	Yes	No
pt_obs	observation maximum value time	Yes	No
pt_mod	simulation maximum value time	Yes	No

pt_err	maximum value time delta	Yes	No

Tips and troubleshooting

NWM forecast download and processing

- Downloading and processing NWM forecasts (the first step, `fetch_fcst_data`) can be highly resource-intensive and time-consuming. For context, retrieving one month of short-range forecasts may take several hours or longer, depending on the number of processors available in your computing environment. Therefore, it is advisable to plan accordingly and allocate sufficient time for this task—for example, by running it overnight.
- If you see the following warnings or errors, ignore them and wait for the actual processing to get started

WARNING:google.auth.compute_engine._metadata:Compute Engine Metadata server unavailable on attempt 3 of 5. Reason: HTTPConnectionPool (host='metadata.google.internal', port=80): Max retries exceeded with url: /computeMetadata/v1/instance/service-accounts/default/?recursive=true (Caused by NameResolutionError("<urllib3.connection.HTTPConnection object at 0x78316c2ede10>: Failed to resolve 'metadata.google.internal' ([Errno -2] Name or service not known"))

- If you encounter warning messages indicating that the system is running low on memory, it’s best to terminate the job to free up resources. You can then restart the job, and the data processing will resume from where it left off.

2025-03-10 10:28:06,101 - distributed.worker.memory - WARNING - Worker is at 80% memory usage. Pausing worker. Process memory: 1.49 GiB -- Worker memory limit: 1.86 GiB